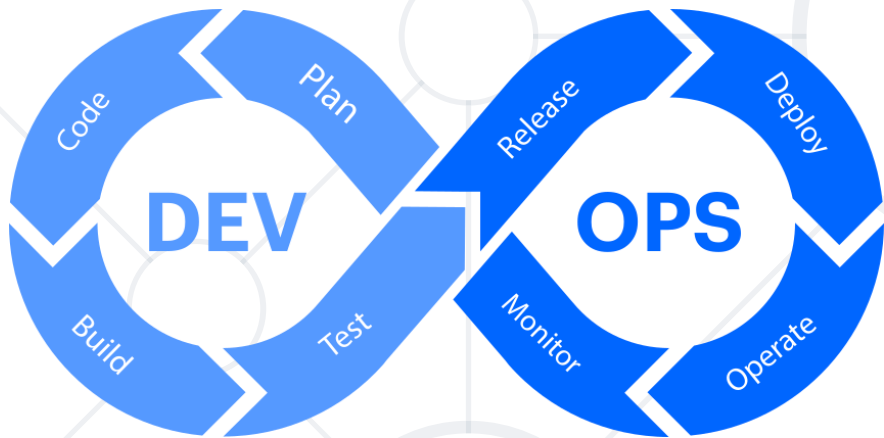


# Introduction and Tool Landscape

The Beginning of Our Journey



**SoftUni Team**  
Technical Trainers



**SoftUni**



Software University

<https://softuni.bg>

[sli.do](https://sli.do)

**#DevOps-CI-CD**

[facebook.com/groups/  
cicdmonitoringjanuary2026](https://facebook.com/groups/cicdmonitoringjanuary2026)



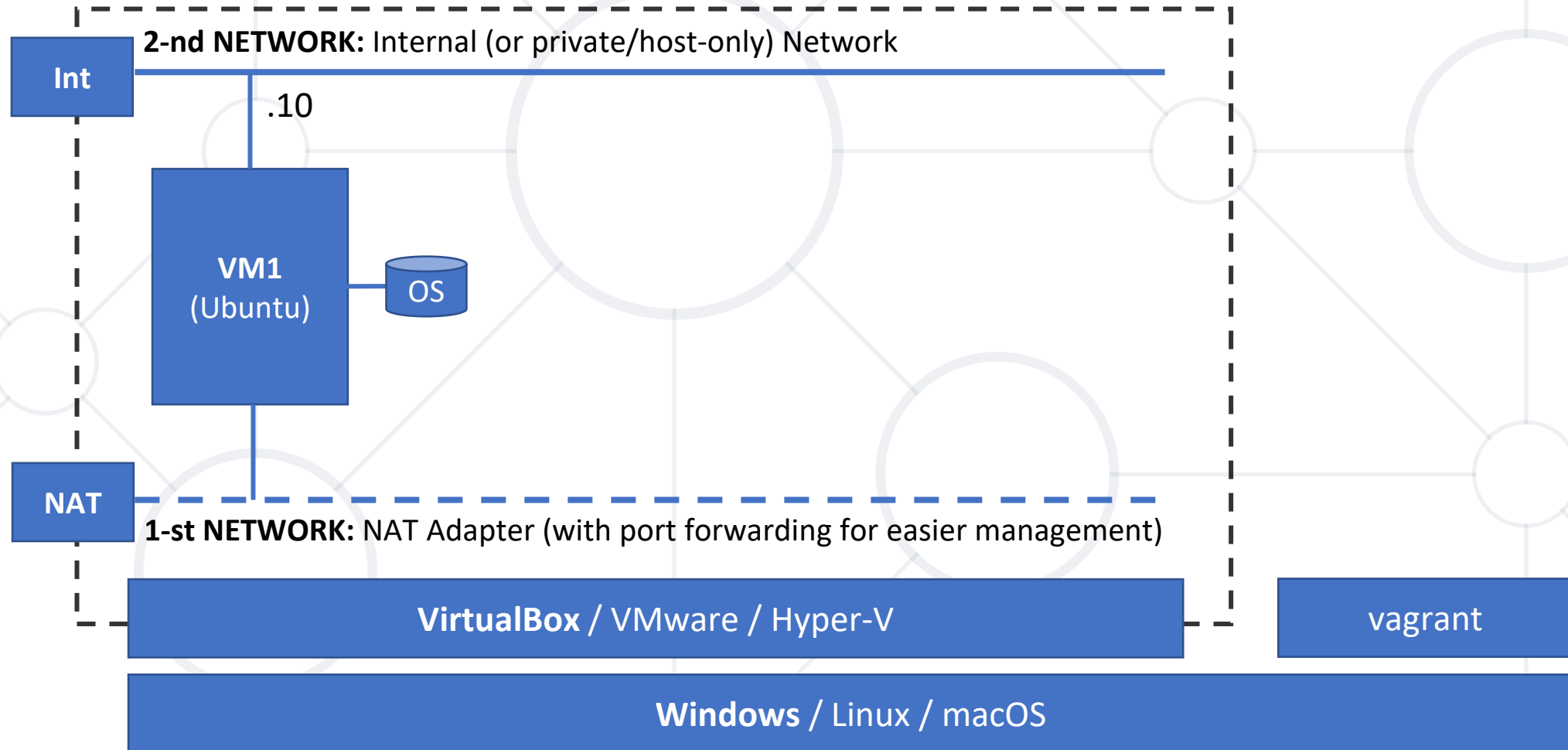
# **This Module (M1)**

## **Topics and Infrastructure**

# Table of Contents

1. SDLC, DevOps, and CI/CD
2. Tooling Landscape
3. Our Roadmap
4. Start Simple







# **SDLC, DevOps and CI/CD**

## **SDLC Overview**

- It is a structured process that guides software development projects from start to finish
- The whole process is broken into **manageable phases or stages** to meet customer expectations and project goals
- It **provides just the framework** for managing complexity, reducing risk, ensuring consistency, and aligning development with business objectives
- **Various models offer different approaches** to these stages

# The Core SDLC Phases\* (1)

- **Planning**
  - Defining project scope, resource allocation, and cost-benefit analysis
- **Requirements Analysis**
  - Gathering user needs and documenting them in a Software Requirement Specification (SRS)
- **System Design**
  - Creating the architectural blueprint, including database design, UI/UX, and API structures

\* Depending on the source and context those vary from 5 or 6 up until 8 or even 10 phases

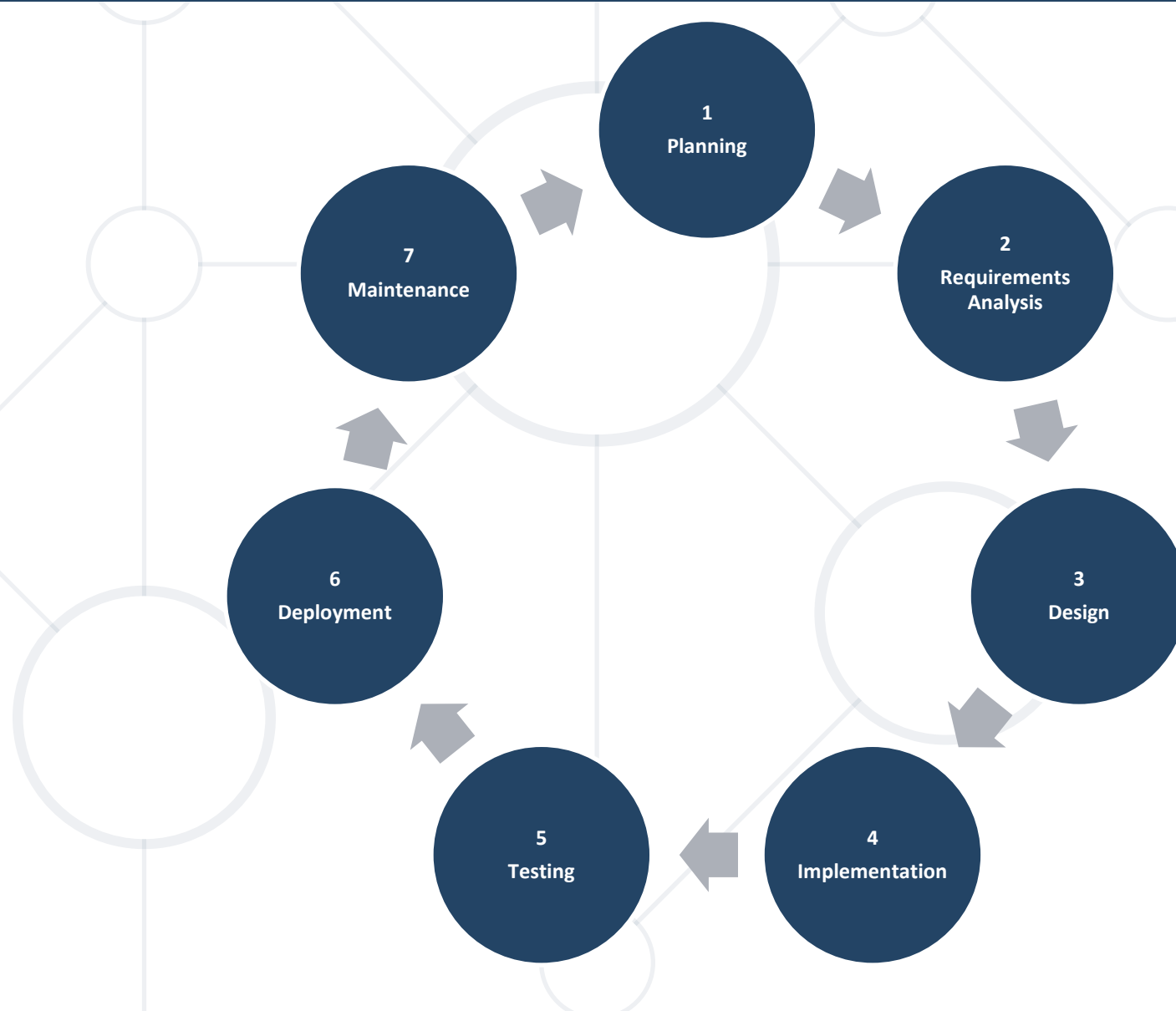


# The Core SDLC Phases\* (2)

- **Implementation (Coding or Development)**
  - The actual writing of code
- **Testing**
  - Rigorous verification to identify bugs and security vulnerabilities
- **Deployment**
  - Moving the code to a production environment
- **Maintenance**
  - Ongoing support, bug fixes, and updates after deployment

\* Depending on the source and context those vary from 5 or 6 up until 8 or even 10 phases

# SDLC Visualized



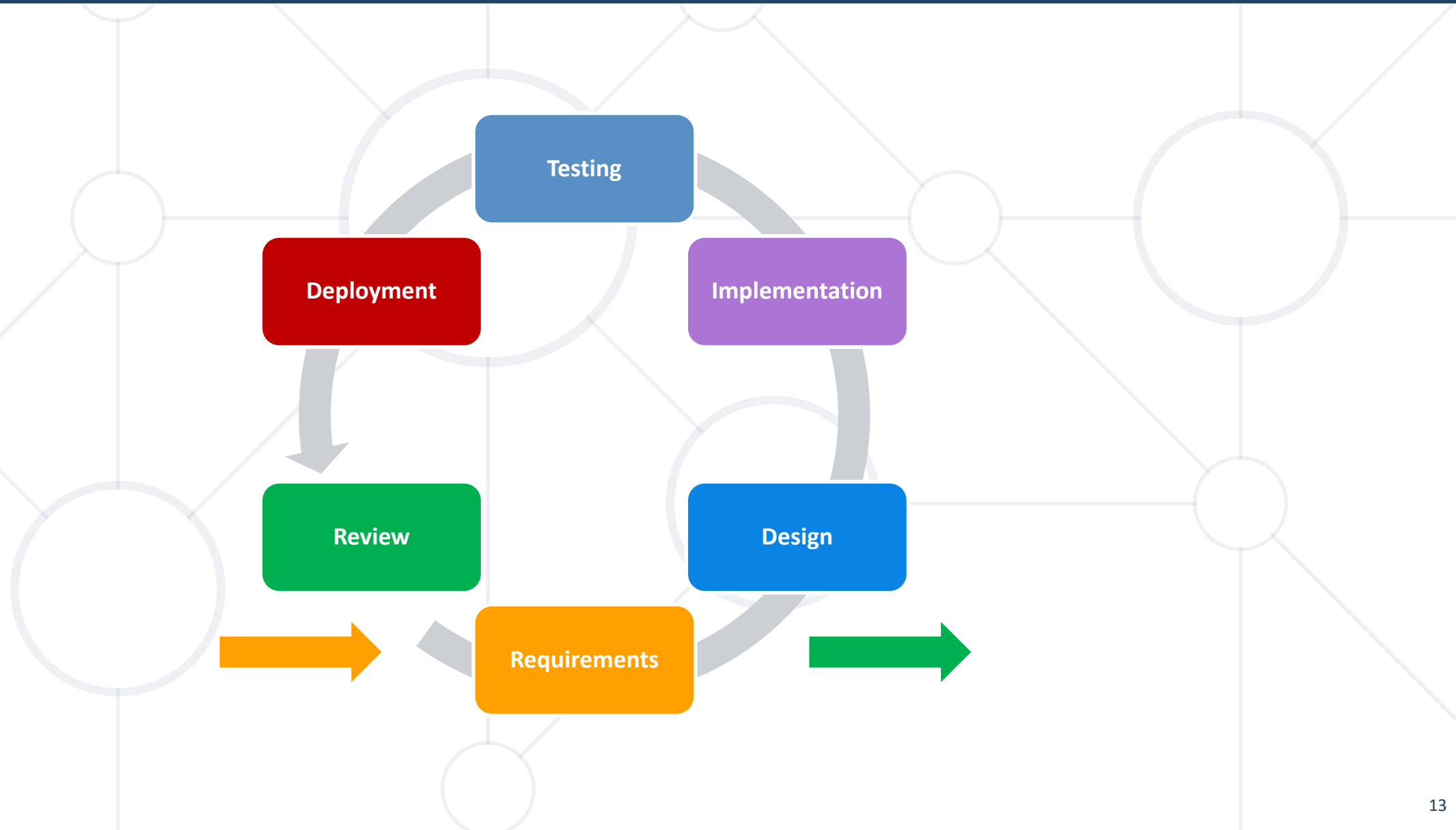
- **Waterfall**
  - Linear and sequential. Each phase must be completed before the next
  - Suitable for small projects with fixed, clear requirements
- **V-Model (Verification and Validation)**
  - An extension of Waterfall that emphasizes a testing phase (verification) for every development phase (validation)
  - Suitable for highly reliable systems needing rigorous testing
- **Incremental (Iterative)**
  - Delivers software in small, functional increments, adding features over time
  - Suitable for getting a working product out quickly and adding features later

## ■ Agile

- Iterative and flexible, focusing on collaboration, customer feedback, and quick, incremental releases
- Suitable for projects with evolving requirements and a need for speed

## ■ DevOps

- Integrates development (Dev) and operations (Ops) through automation and collaboration to shorten the development lifecycle
- Other models often treat development and operations as separate phases or handoff points
- Suitable for continuous delivery and faster deployment





# **SDLC, DevOps and CI/CD**

## **The Shift**

- **Everything changes** with time, including customer needs and demands
- Key ones are **faster innovation**, a **growing customer base**, **cost-effectiveness**, **stability**, etc.
- **We cannot** address today's challenges with yesterday's technology
- This leads to changing the software and the way it is developed
- It is about the shift from **monolith (*linear SDLC*)** to **microservices architecture (*asynchronous SDLC*)**
- It is about moving from a single codebase with single deployment unit, to **multiple decoupled services** communicating via APIs
- But it is also, to how the teams are **organized and communicate**

Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure.

Melvin Conway, 1967

- The architecture of the final product reflects how the teams are organized and communicate
- If teams are siloed, the system will have tight dependencies
- If teams are cross-functional, the system will likely be more modular, with different components designed in parallel

\* [https://www.melconway.com/Home/Conways\\_Law.html](https://www.melconway.com/Home/Conways_Law.html)



# The Two-Pizza Team Rule

“We try to create teams that are no larger than can be fed by two pizzas.”  
Jeff Bezos

- Size matters. Aim for 5-10 people
- Better agility and speed. Smaller teams communicate faster and make quicker decisions, essential for innovation
- Ownership and accountability. Small, focused teams have clear responsibility for delivering business value, increasing individual contribution
- Fewer people mean less coordination, fewer meetings, and less wasted time
- Teams are self-sufficient and have all necessary skills to deliver their product or service

- A framework for organizing software delivery teams around business value streams to improve flow, reduce silos, and accelerate delivery
- Minimizes cognitive load and aligns teams with architecture
- Four key team types
  - **Stream-aligned Teams:** Core teams focused on a single end-to-end business flow
  - **Enabling Teams:** Help stream-aligned teams adopt new technology or practices
  - **Complicated Subsystem Teams:** Handle complex, specialized components needing deep expertise
  - **Platform Teams:** Provide internal platforms to accelerate other teams
- Three interaction models
  - **Collaboration:** Joint effort for a limited time to solve a specific problem
  - **X-as-a-Service:** One team provides a service for others to consume
  - **Facilitation:** Enabling teams guide others to build capabilities themselves

- A **fundamental organizational change** moving from siloed teams to a collaborative and shared-responsibility model
- With a focus on **transparency, automation, continuous learning, and fast feedback** to deliver software faster and more reliably
- It's about **breaking down barriers, fostering empathy, and empowering teams** with shared ownership over the entire software lifecycle
- Focusing on **customer value** and **continuous improvement** rather than just tools or processes

- **Culture:** Breaking down silos between Development and Operations, fostering shared responsibility, open communication, and trust
- **Automation:** Automating repetitive tasks in the software lifecycle (build, test, deploy) to reduce errors and increase speed
- **Lean:** Applies lean principles to eliminate waste, focus on customer value, and improve the flow of work
- **Measurement:** Collecting and analyzing data on performance, feedback, and system metrics to drive continuous improvement
- **Sharing:** Open communication, feedback loops, and sharing knowledge, tools, and best practices across teams
- **Security (DevSecOps):** Integrating security practices throughout the entire lifecycle, not just at the end

- **Continuous Integration (CI): Automated building and testing**
- **Continuous Delivery (CD):** Every build is a **candidate**. One click pushes it to production
- **Continuous Deployment (CD):** Zero human intervention. If **code passes tests, goes to production**
- CI/CD **automates** the core steps of the Software Development Lifecycle (SDLC) like **building, testing, and deploying code**
- It provides the **automated tools** and **pipelines** that bring the DevOps philosophy of continuous improvement to life within the SDLC's stages



**Break**

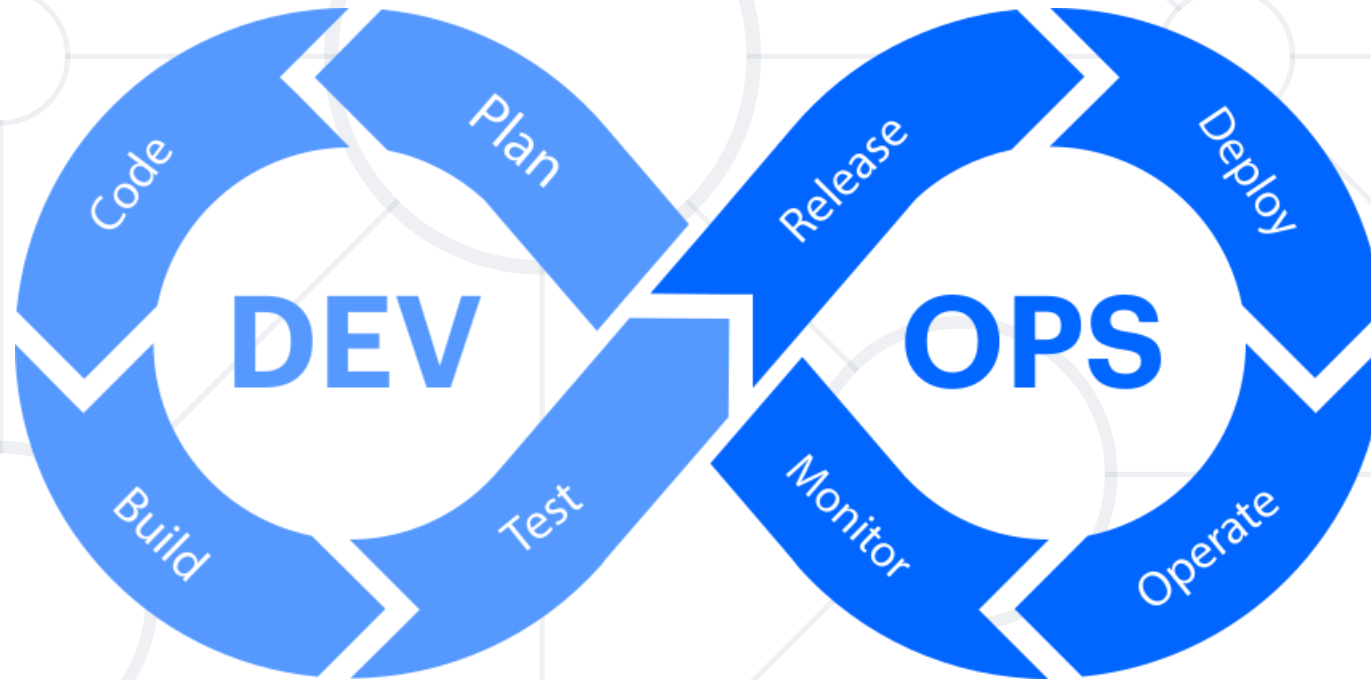
**See you in a moment 😊**



# Tooling Landscape

## A Different View

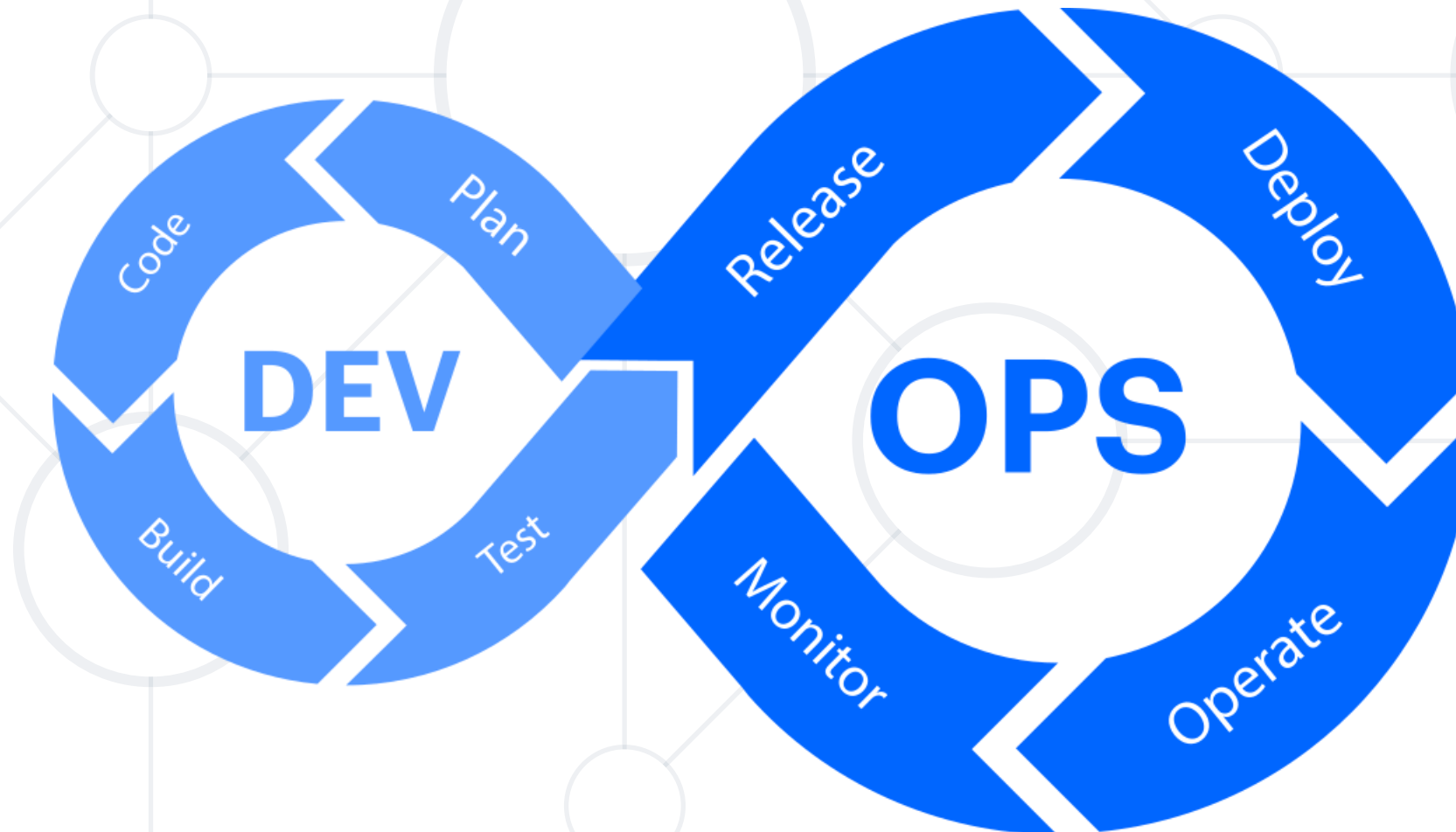
- Usually, we use a picture like this to discuss and explain DevOps





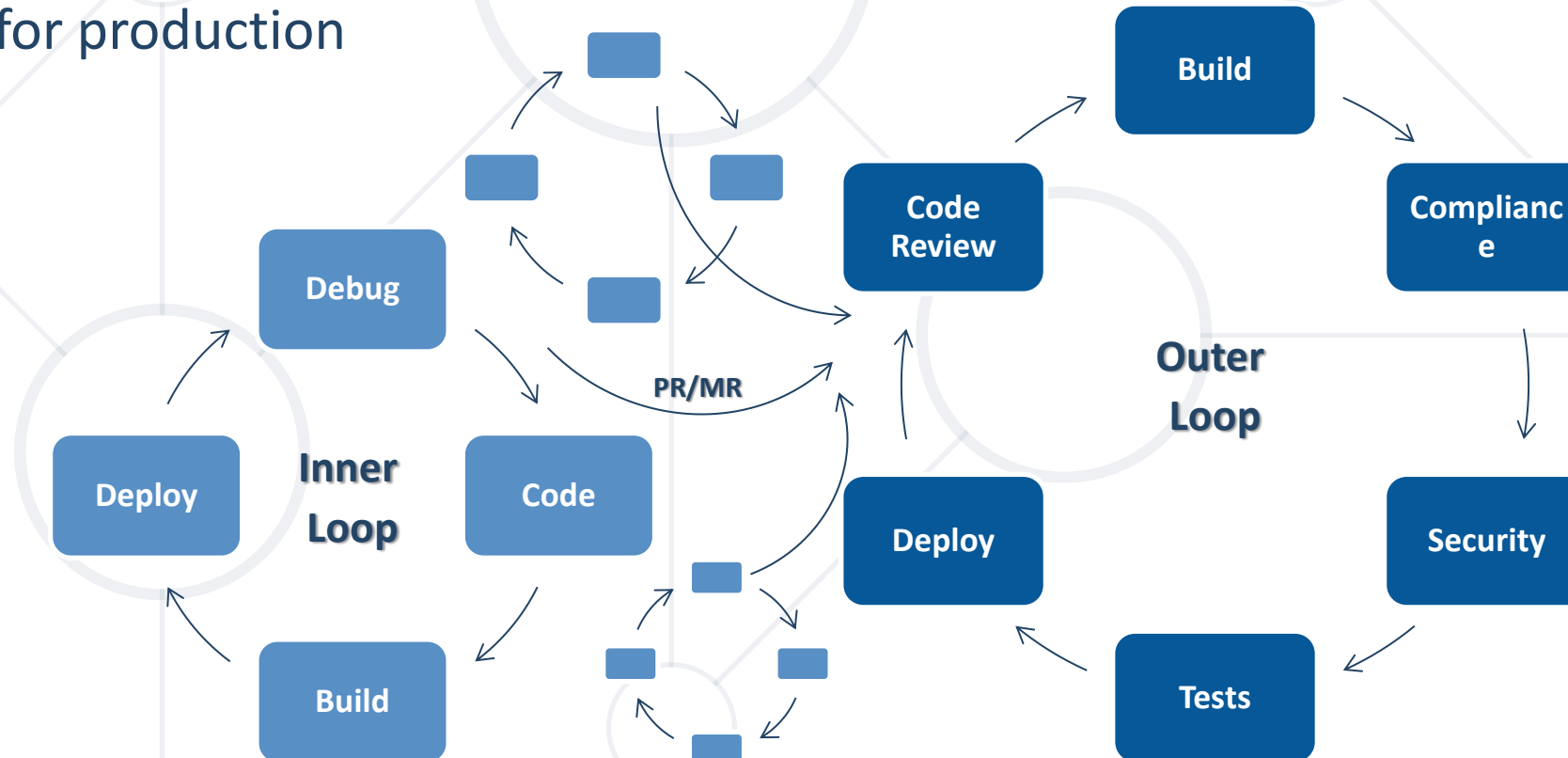
# DevOps Continuity: Different Perspective

- If we take into consideration **time, speed, involvement, visibility,** etc., we will end up with a different picture



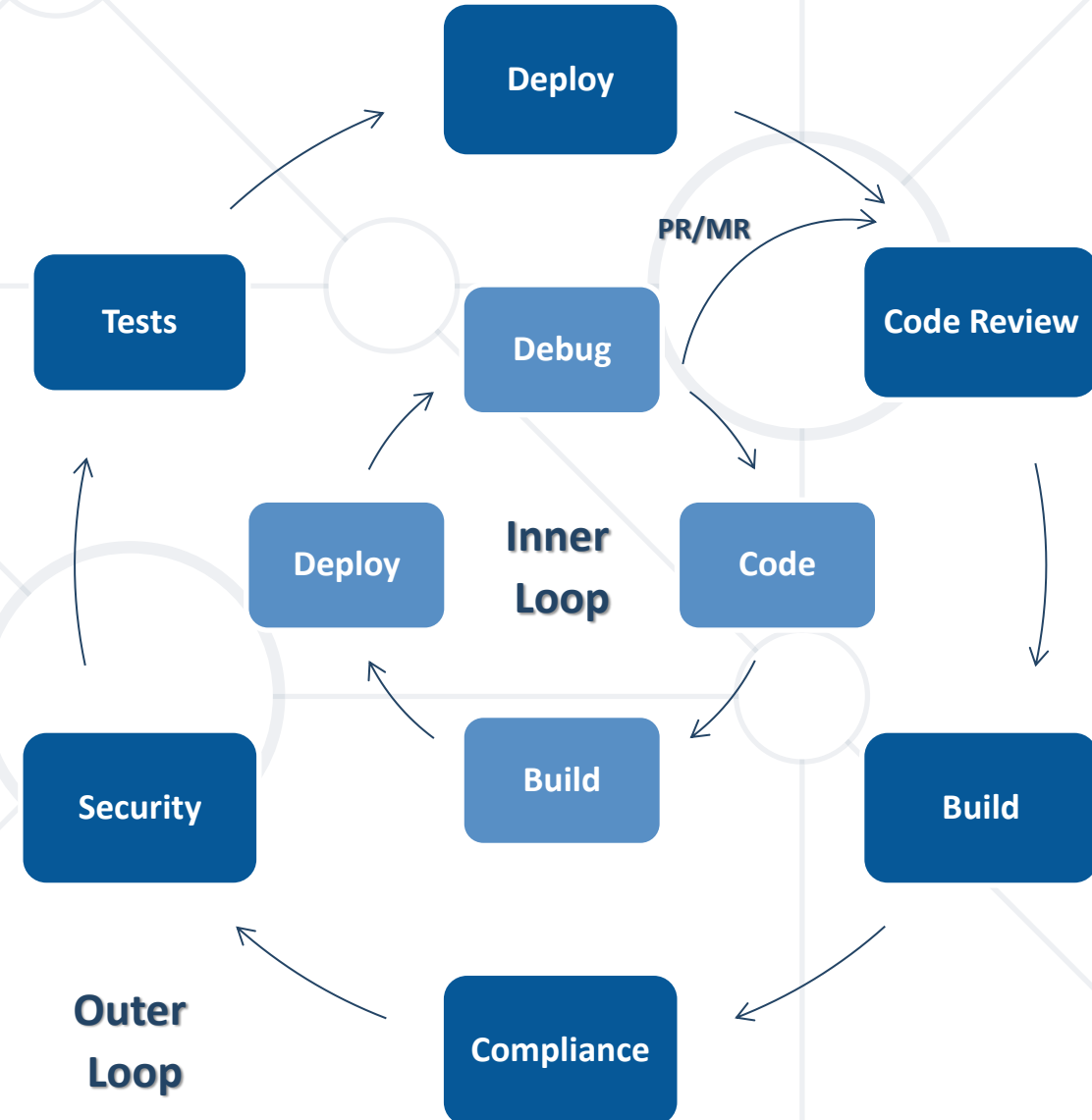
# The Two Loops

- The **Inner Loop** is the *rapid, developer-focused* cycle of coding, building, and local testing (minutes/hours), providing quick feedback on changes
- The **Outer Loop** is the *slower, team-wide* process involving code reviews, integration, security checks, and full CI/CD deployment (hours/days/weeks), ensuring code quality and readiness for production



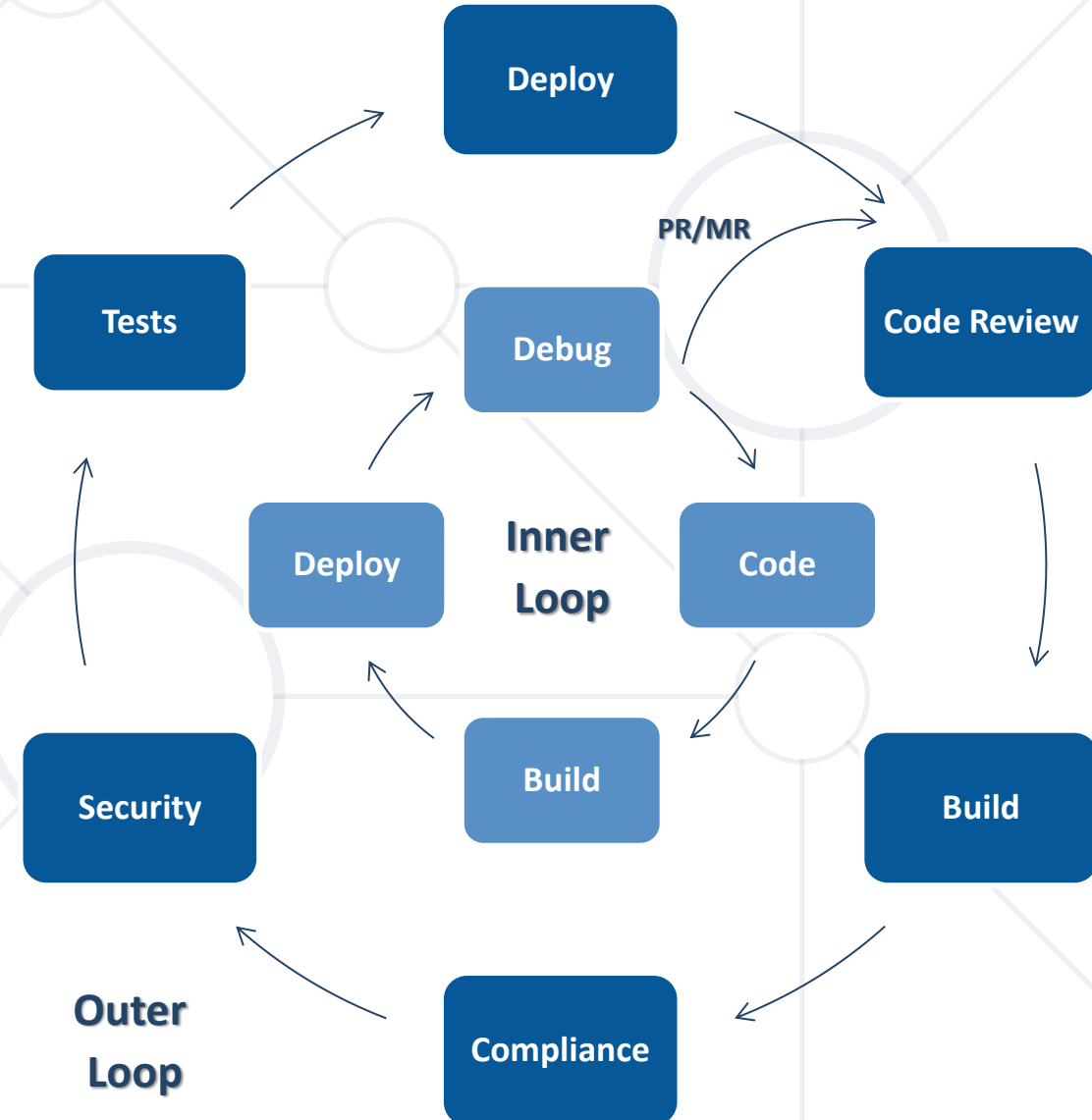
# Inner Loop (Developer's World)

- **Focus:** Individual developer productivity and immediate feedback
- **Activities:** Writing code, building, running, debugging, unit testing, committing code.
- **Environment:** Mostly local machine, fast feedback
- **Goal:** Quickly validate code changes



# Outer Loop: Team & Release World

- **Focus:** Team-level quality, integration, security, and deployment
- **Activities:** Code reviews, integration testing, security/compliance scans, full CI/CD pipelines, deployment to staging/production, data provisioning, requirements engineering
- **Environment:** Shared servers, CI/CD systems, container environments (Kubernetes)
- **Goal:** Deliver robust, reliable, production-ready software



# Inner and Outer Loop Summary

- A **code commit** to source control often marks the **handoff** or the **transition from the inner to the outer loop**
- Inner loop **friction** slows **individual developers**; outer loop friction slows the **entire team and delivery**
- **Optimizing both is crucial**. Fast inner loops boost individual productivity, while streamlined outer loops reduce friction and speed up releases
- For the ***inner loop***, we should strive **for parity between local and production environment**. This could be achieved by ephemeral environments using tools like Tilt, Skaffold, Garden, etc. to live-reload code directly into a local Kubernetes cluster
- For the ***outer loop***, we could use various techniques and tools. For example, on CI level, we could **implement fast-fail logic** by running cheapest tests first, and on CD level, we could adopt **various deployment strategies**

A dark blue circle containing a grey gear. The word "Dev" is in white and "Ops" is in yellow, both in a bold sans-serif font.

**DevOps**

**Tooling Landscape**  
**Tools**

# The DevOps Tools Periodic Table\*

- Organizing the chaos of the DevOps ecosystem by categorization of the tools by lifecycle stage
- **Plan & Code**
  - Jira, Confluence, Git (GitHub, GitLab, Bitbucket)
- **Build & Test (incl. Security)**
  - Jenkins, GitLab CI, GitHub Actions, Circle CI, SonarQube, Snyk, OWASP ZAP
- **Release & Deploy**
  - Spinnaker, Octopus Deploy, ArgoCD, FluxCD, Tekton, Harness
- **Operate & Monitor**
  - Prometheus, Grafana, Elastic Stack, OpenTelemetry, Zabbix, Splunk

\* <https://digital.ai/learn/devsecops-periodic-table/> | <https://landscape.cncf.io/> | <https://roadmap.sh/devops>

# Choosing the Right Stack\*

## ■ The Decision Matrix

| Feature     | All-in-One (e.g., GitLab)    | Best-of-Breed (e.g., Jenkins + Jira + Spinnaker) |
|-------------|------------------------------|--------------------------------------------------|
| Integration | Seamless, it just works      | Complex, requires custom API work/plugins        |
| Flexibility | Limited to platform features | Infinite, swap tools as needed                   |
| Maintenance | Low, one vendor to manage    | High, multiple updates and licenses              |
| Governance  | Centralized permissions      | Fragmented, requires SSO/IAM synchronization     |

## ■ The Gravity Principle

- Choose tools that have the most gravity (community support and documentation)
- If you get stuck at 2 AM, can you find a solution on StackOverflow or via AI

## ■ Avoiding Tool Sprawl

- Every tool you add is a liability
- If a tool doesn't solve a specific bottleneck identified by your Value Stream Map, don't adopt it



- **The "Pet vs. Cattle" Paradigm**
  - **Pets:** Servers you name, nurture, and fix manually (Stop doing this!)
  - **Cattle:** Identical instances that are replaced, not repaired
- **Declarative vs. Imperative**
  - **Imperative (How):** Run **apt-get install**, then **service start**. High risk of *Configuration Drift*
  - **Declarative (What):** I need 3 Nginx containers running on Port 80. The system handles the **how**
- **Bottomline**
  - Manual work is **error-prone**. If you do it a few times, **automate it**



# Tooling Landscape

## A Few Trends

## ■ The Reality

- DevOps overload – modern DevOps is too complex for one human
- A developer shouldn't need to be a Kubernetes, Cloud, and Security expert simultaneously

## ■ The Solution

- Internal Developer Platform (IDP) – a layer built by Platform Engineers that wraps the complexity
- Self-service – Developers define workloads (CPU, Mem, Port) and the Platform handles the underlying infrastructure

## ■ The Golden Path

- Creating a paved road that is so easy to follow, developers choose it over **Shadow IT**

- **Concept**

- Managing infrastructure and application state via Git repositories
- Git as the **Source of Truth** – If it's not in Git, it doesn't exist

- **Mechanism**

- Define the **desired** application or infrastructure **state** in a Git repo
- An **agent watches that repo**
- If the desired state in Git differs from the actual state in the target environment, the agent **heals it back** to what is in Git

- **Benefit**

- Auditability and **instant disaster recovery**
- You know exactly **who changed what and when** for the entire environment

- **From Tools to Teams**

- Moving from **Copilots** to **Autonomous Agents** – transition from simple code-completion tools to **Multi-Agent Systems**

- **The Agentic Workflow**

- **Planning Agent:** Analyzes requirements and drafts architecture
- **Coding Agent:** Implements features and writes tests in parallel
- **QA/Security Agent:** Validates code against security policies and performance baselines

- **Key Benefit**

- Agents now manage **git worktrees** and **remote environments** independently, allowing humans to act as **Orchestrators** rather than manual coders



**Break**

**See you in a moment 😊**



## **Our Roadmap**

**A journey of a thousand miles begins with a single step**

# The Journey from Code to Insight

- **Phase 1: Foundations**
  - Continuous integration, Git flow, build triggers, and artifact versioning
- **Phase 2: Quality & Delivery**
  - Incorporate testing and deployment strategies
- **Phase 3: Security & Advanced Ops**
  - Incorporate security and move to GitOps
- **Phase 4: Observability & Monitoring**
  - Get performance and behavior insights

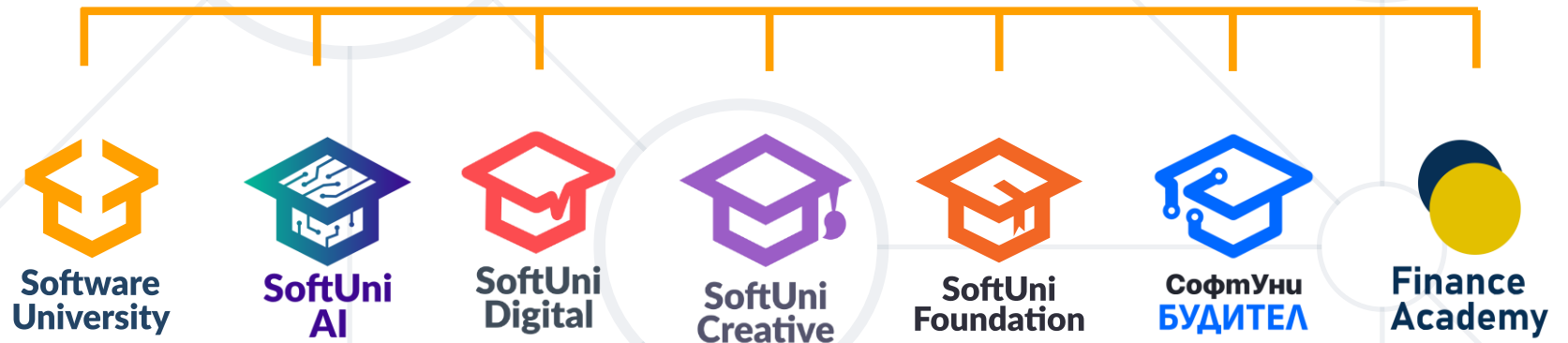


- Avoid the over-engineering trap that kills DevOps initiatives
- **The Minimum Viable Pipeline (MVP):**
  - **Step 1:** Code is pushed to a feature branch
  - **Step 2:** A linter checks for syntax errors (Fast failure)
  - **Step 3:** The app is containerized (Docker)
  - **Step 4:** Deploy to a sandbox environment
- **The Rule of Three**
  - Do not automate a process until you have done it manually three times. Understand the pain before you write the script
- **Metrics of Success**
  - Your goal for Week 1 isn't zero bugs - it's reducing Lead Time for Change - how fast can you get a 1-line change from your laptop to a live URL



**Practice: Start Even Simpler**  
**Let's Spin an App or Two**

# Questions?



- Software University – High-Quality Education, Profession and Job for Software Developers

- [softuni.bg](http://softuni.bg), [about.softuni.bg](http://about.softuni.bg)

- Software University Foundation

- [softuni.foundation](http://softuni.foundation)

- Software University @ Facebook

- [facebook.com/SoftwareUniversity](https://facebook.com/SoftwareUniversity)



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg>
- © Software University – <https://softuni.bg>

